Student instructions: running the vascular extraction application on the Xeon + A2 PC

This runs one experiment, the baseline with the built-in prompt, from Eclipse. It also records speed and resource data. I wrote the file paths from the locations you gave me. Where I had to choose a path myself, it is marked **(choose)**.

What you are running: a Java program that starts a local LLM server (llama-server.exe), sends each radiology report to the model, and asks it to return the carotid velocities, % stenosis and ICA/CCA ratios as JSON. It writes the results to tab-separated files, along with timing data. The program starts and stops the server itself.

Part A. One-time preparation

Step 1. Install the latest source code in Eclipse

Do:

1. In Eclipse, close the project's open editors. In Windows Explorer, copy C:\Users\VHATVHGOBBEG\ArtVelJavaToLlamaServer\src to a backup folder such as C:\Users\VHATVHGOBBEG\ArtVelJavaToLlamaServer_src_backup.
2. Unzip the most recent src.zip (30 .java files in 9 package folders) into C:\Users\VHATVHGOBBEG\ArtVelJavaToLlamaServer\src. Choose "overwrite all".
3. Make sure no .java files sit loose in the src root. Delete any that do. The older flat-layout copies are obsolete and cause duplicate-class errors.
4. In Eclipse, click the project and press **F5** (Refresh). Then choose **Project > Clean... > Clean all projects > Clean**.

Why: The batch runner is in the newest zip. Cleaning forces a full rebuild.

Check: The Problems view shows no errors, and src\core\VascularBatchRunner.java exists. The packages should be config, core, database, extract, locate, model, output, server, telemetry.

Step 2. Confirm the Java runtime

Do: Right-click the project > **Properties > Java Build Path > Libraries**. Confirm a JRE or JDK of version 19 or newer is listed, along with the Jackson jars (the project already builds, so they should be there).

Why: The code needs Java 19 or newer and Jackson for JSON.

Step 3. Check the llama-server folder

Do:

1. Open C:\Users\VHATVHGOBBEG\LLM\Server\. It must contain the **whole contents of the llama.cpp download**, not just llama-server.exe. This means all the .dll files, including the CUDA ones (names like ggml-cuda.dll, cudart64_12.dll and cublas64_12.dll).
2. Open a Command Prompt and run:

```
nvidia-smi
```

It should list the NVIDIA A2.
3. Run:

```
"C:\Users\VHATVHGOBBEG\LLM\Server\llama-server.exe" --help | findstr /i "load-mode mlock"
```

Why:

- Without the DLLs, the server starts but runs on the CPU only, or fails to start.
- nvidia-smi shows that the GPU driver works.
- Step 3.3 tests the one unusual setting in the settings file, server.extra-args = --load-mode mlock. This setting asks Windows to keep the model in RAM and not page it out. If neither --load-mode nor --mlock appears in the output, your server build doesn't support them. In that case, delete the server.extra-args line in Step 5.

Step 4. Smoke-test the server and model by hand (about 5 minutes)

Do:

1. In Command Prompt run:

```
"C:\Users\VHATVHGOBBEG\LLM\Server\llama-server.exe" -m "C:\Users\VHATVHGOBBEG\LLM\Models\gemma-4-E4B-it-IQ4_NL.gguf" -ngl 999 --port 8080
```

2. Wait until the window prints that the server is listening. The model loading can take a minute or more.
3. In a web browser open http://127.0.0.1:8080. A chat page should appear. Type "hello" and confirm you get a reply.

4. While it is running, run nvidia-smi in a second window and confirm GPU memory use rose by a few GB.
5. Go back to the server window and press **Ctrl+C** to stop the server. Close the window.
6. If Windows Firewall asks about llama-server.exe, allow it on **private networks** only. The application talks to the server only on the same computer (127.0.0.1).

Why: This shows that the model, the GPU and the server work, before any of our code is involved. If this step fails, nothing else will work.

Check: Make sure no llama-server.exe is left running. Open Task Manager > Details and end any that remain. Also check that nothing else uses port 8080. In Command Prompt, netstat -ano | findstr :8080 should print nothing.

Step 5. Edit the machine settings file

File: C:\Users\VHATVHGOBBEG\LLM\configuration\properties\vascular-va-xeon-a2.properties

This file describes this computer. Open it in Notepad or Eclipse. Backslashes are fine, and no quotes are needed. Change these lines (find each by its key):

server.executable = C:\Users\VHATVHGOBBEG\LLM\Server\llama-server.exe

server.model = C:\Users\VHATVHGOBBEG\LLM\Models\gemma-4-E4B-it-IQ4_NL.gguf

In the # ---- corpus section:

corpus.directory = C:\Users\VHATVHGOBBEG\LLM\corpus_test

corpus.extension = .txt

(choose) Use a small folder at first (see Step 6).

In the # ---- experiment runs section:

output.base-directory = C:\Users\VHATVHGOBBEG\LLM\runs

run.machine-name = VA-Xeon-A2

run.hardware-notes = Xeon Silver 4310, 12 cores, Nvidia A2 16 GB, 128 GB RAM

Replace 128 GB with the real RAM size. **(choose)** The runs folder doesn't need to exist.

Leave everything else as it is. If Step 3.3 showed that your server doesn't support --load-mode, delete the line server.extra-args = --load-mode mlock or change it to server.extra-args = --mlock.

Why: These lines point the program at the server, the model, the reports to read, and where to put results. The settings that affect speed on this machine are already in the file (all layers on the GPU, 4 parallel slots, a 49152-token context shared across the slots). Don't change them for the first run.

Note: The run folders contain text derived from the reports. Keep output.base-directory on storage that is approved for that data.

Step 6. Prepare the reports

Do:

1. Create C:\Users\VHATVHGOBBEG\LLM\corpus_test and put **5 reports** in it as .txt files, one report per file.
2. Later, when the test works, you will point corpus.directory at the full folder of 100 reports.

Why: A 5-report trial takes a few minutes. If a path or setting is wrong, you find out quickly.

Step 7. Check the experiment file

File: C:\Users\VHATVHGOBBEG\LLM\configuration\properties\experiment-baseline.properties

No edits are needed for the first run. It sets the label baseline, uses the prompt built into the code, and uses deterministic sampling (temperature 0, top-k 1). It leaves prompt.file commented out on purpose. The prompt-builtin.txt file is a copy of the built-in prompt. You only need it when you want an edited prompt later.

Why: The experiment file holds what you vary between runs. The machine file holds what stays the same on this computer.

Step 8. Edit the batch file

File: C:\Users\VHATVHGOBBEG\LLM\configuration\batch_runs\batch-example.txt

Replace the entire contents with:

First run: baseline, once

C:\Users\VHATVHGOBBEG\LLM\configuration\properties\experiment-baseline.properties

Why: The example lists the experiment files by bare name, and the program looks for them in the batch file's own folder. Yours are in a different folder, so the full path is needed. The variant experiment is removed because it needs an edited prompt file that doesn't exist yet.

Part B. Create the Eclipse run configuration

Step 9

Do:

1. **Run > Run Configurations...** Right-click **Java Application > New Configuration**.
2. **Name:** Batch - Xeon A2
3. **Project:** ArtVelJavaToLlamaServer. **Main class:** core.VascularRunner
4. Open the **Arguments** tab. In **Program arguments** put everything on one line:

```
RUN_EXPERIMENT_BATCH
C:\Users\VHATVHGOBBEG\LLM\configuration\batch_runs\batch-example.txt --
config=C:\Users\VHATVHGOBBEG\LLM\configuration\properties\vascular-va-xeon-
a2.properties
```

5. Leave **VM arguments** empty. Click **Apply**. Don't click Run yet.

Why: The first argument tells the program to run a batch without showing menus or prompts. The second is the batch file. --config= selects this computer's settings file.

Part C. Run it

Step 10. Pre-flight checklist

- No llama-server.exe appears in Task Manager.
- Nothing is using port 8080 (see Step 4).
- Windows power plan is **High performance** and the PC will not go to sleep. The first time you do this, also close other GPU-heavy programs.
- Free disk space is at least a few GB.

Step 11. Start the resource monitor (optional but recommended)

Do:

1. Open a Command Prompt, separate from Eclipse.
2. Run:

```
cd C:\Users\VHATVHGOBBEG\LLM\configuration\cpu_resource_monitors

sample_resources_windows.bat
C:\Users\VHATVHGOBBEG\LLM\runs\xeon_resources.csv 5
```

3. Leave the window open. It prints that it found nvidia-smi and that it is sampling every 5 seconds.
4. First time only, open xeon_resources.csv after about 20 seconds to confirm there are rows. At this point the llama-server columns are empty, which is correct because the server isn't running.

Why: It logs CPU, RAM, GPU use and GPU memory every few seconds, so you can see how hard the machine works. Without it you only get timing.

Step 12. Run the application

Do:

1. In Eclipse, click **Run** on the Batch - Xeon A2 configuration.
2. Watch the Console. You will see in order: the settings summary, a line like Experiment run 261101_... (5 documents), the server starting (the model load can take a minute), then one line per document, then a summary.
3. Don't click the red Stop button and don't close Eclipse while it runs.

Why: The program starts the server, processes the reports and shuts the server down at the end. If you kill the process abruptly, llama-server.exe may stay running with the model loaded. Always check Task Manager afterward and end any leftover llama-server.exe before the next run.

Expected: On 5 documents, it finishes in a few minutes and the console ends with Batch ... finished: 1 of 1 run(s) completed.

Step 13. Stop the resource monitor

Click the monitor's Command Prompt window and press **Ctrl+C**, then **Y** if it asks.

Part D. Check the results

Look in C:\Users\VHATVHGOBBEG\LLM\runs\. There should be:

- runs.csv, with one row per run: speed, tokens and the settings used.
- batch_<id>_status.csv, with one row per run and a status of COMPLETED or ABORTED.
- A run folder named yyMMdd_HHmmss_VA-Xeon-A2_gemma-4-E4B-it-IQ4_NL_baseline.

Inside the run folder:

- summary.json has the totals: documents per minute, median and 95th-percentile time per document, and counts of ok, failed and too-long documents.
- ..._documents.tsv has one row per document. Check the status column.
- The other result files are the three extracted-value files, which list the ICA/CCA ratios, the flow velocities and the stenosis percentages.
- raw_responses\ holds the model's raw answer for each document, and server.log and results.log hold the logs.
- manifest.json records everything needed to reproduce the run, including the settings and the SHA-256 hash of the prompt.

First-run sanity check: Open documents.tsv. All 5 documents should have a status that is not an error. Open one raw response and one report side by side and confirm the extracted values match the report text.

Part E. Scaling up and troubleshooting

Scale to 100 reports: Change corpus.directory in the machine file to the full folder, then repeat Parts C and D. Nothing else changes.

If something goes wrong

Symptom	Likely cause and fix
Console shows a settings error naming a file or key	Typo or wrong path in a .properties file. The message names the key.
Run is ABORTED and server.log mentions the port or a failed bind	Another llama-server.exe is still running. End it in Task Manager and try again.

Symptom	Likely cause and fix
server.log mentions CUDA, a missing DLL, or the model fails to load	The DLLs aren't in the server folder (Step 3).
Server never reaches "listening"	Check the model path. Increase server.startup-timeout-minutes.
Many documents have status TOO_LONG	The reports are too long for the slot size. The program retries those in a second pass automatically.
notes column contains REASONING_CONTENT	The model began "thinking" instead of answering. In experiment-baseline.properties, change generation.use-grammar = false to true.
Very slow, and nvidia-smi shows little GPU use	The server is running on the CPU. Check server.n-gpu-layers = 999 and the DLLs.

When you finish a run, send me or your supervisor the run folder, runs.csv and xeon_resources.csv for comparison.